ParetoEnsembles.jl: A Julia Package for Multiobjective Parameter Estimation Using Pareto Optimal Ensemble Techniques

Jeffrey D. Varner*
Robert Frederick Smith School of Chemical and Biomolecular Engineering
Cornell University, Ithaca, NY 14853

Abstract

Mathematical models of biological and engineering systems often have many adjustable parameters that must be estimated from multiple, potentially conflicting datasets. Rather than reporting a single best-fit parameter vector, it is often more informative to generate an *ensemble* of parameter sets that collectively map out the trade-offs among competing objectives. ParetoEnsembles.jl is an open-source Julia package that generates such ensembles using Pareto Optimal Ensemble Techniques (POETs), a simulated-annealing-based algorithm that requires no gradient information and is defined entirely by four user-supplied callback functions. This paper describes a substantially rewritten implementation that corrects the original dominance relation, reduces the per-iteration ranking cost through an incremental update scheme, and adds multi-chain parallel execution for improved front coverage. We demonstrate the package on two standard benchmark problems, a cell-free gene expression model fitted to experimental data, and a blood coagulation cascade model with ten estimated rate constants and three objectives. A head-to-head comparison with NSGA-II shows that ParetoEnsembles.jl produces denser fronts with comparable or superior hypervolume and inverted generational distance, and a sensitivity analysis confirms robustness to hyperparameter choices. ParetoEnsembles.jl is available under the MIT license at `https://github.com/varnerlab/ParetoEnsembles.jl`.

Keywords: multiobjective optimization, Pareto front, simulated annealing, ensemble methods, Julia, parameter estimation

1 Introduction

Mathematical models of biological and engineering systems typically contain many adjustable parameters that must be estimated from experimental data. When multiple datasets measure different aspects of the same system — for example, messenger RNA (mRNA) and protein concentrations in a gene expression circuit, or yield and selectivity in a reactor — calibrating parameters to one dataset often degrades the fit to another, giving rise to a multiobjective optimization (MOO) problem [Miettinen, 1999, Deb, 2001, Coello Coello et al., 2007]. Compounding this difficulty, many models exhibit "sloppy" parameter sensitivities: broad, flat valleys in objective space where large regions of parameter space produce nearly equivalent fits [Gutenkunst et al., 2007]. In such settings, reporting a single best-fit parameter vector obscures the practical uncertainty; instead, one needs an *ensemble* of parameter sets that collectively map out the trade-offs among competing objectives. The boundary of this ensemble in objective space is the Pareto front — the set of solutions for

*Corresponding author: `jdv27@cornell.edu`

which no objective can be improved without degrading at least one other — and the full cloud of near-optimal solutions surrounding it characterizes the range of model behaviors consistent with the available data.

Several families of algorithms have been developed to approximate Pareto fronts. Population-based evolutionary methods maintain a generation of candidate solutions that are selected, recombined, and mutated toward the front; prominent examples include the Non-dominated Sorting Genetic Algorithm II (NSGA-II) [Deb et al., 2002], its many-objective successor NSGA-III [Deb and Jain, 2014], and the Multiobjective Evolutionary Algorithm Based on Decomposition (MOEA/D), which decomposes the MOO problem into a set of scalar subproblems solved cooperatively [Zhang and Li, 2007]. An alternative trajectory-based strategy is multiobjective simulated annealing (SA), which extends the classical SA framework [Kirkpatrick et al., 1983] to multiple objectives by using Pareto dominance to guide acceptance decisions; early formulations include the Pareto SA method of Czyżak and Jaszkiewicz [1998] and the Archived Multiobjective Simulated Annealing (AMOSA) algorithm of Bandyopadhyay et al. [2008]. Pareto Optimal Ensemble Techniques (POETs), introduced by Song et al. [2010] and later implemented in the Julia programming language as JuPOETs [Bassen et al., 2017], belongs to this SA-based family; POETs ranks the current archive by counting how many other solutions dominate each candidate, then uses the rank as the energy in a Metropolis-type acceptance criterion. Mature software implementations of these and related algorithms are available in several languages, including pymoo in Python [Blank and Deb, 2020] and Metaheuristics.jl in Julia [Mejía-de Dios et al., 2022], both of which focus primarily on population-based evolutionary strategies.

In this study, we present ParetoEnsembles.jl, a substantially rewritten Julia [Bezanson et al., 2017] implementation of the POETs algorithm. The package takes a user-defined model and a set of objective functions — each measuring how well a candidate parameter vector reproduces a particular dataset — and returns a collection of parameter sets spanning the trade-off surface, along with their objective values and dominance ranks. Several algorithmic corrections and performance improvements have been made relative to the original implementation: the dominance check has been corrected from weak to strict Pareto dominance, the per-iteration ranking cost has been reduced from $O(n^2m)$ to $O(nm)$ through an incremental update scheme (where n is the archive size and m is the number of objectives), and multi-chain parallel execution has been added to improve front coverage. We describe the methods underlying ParetoEnsembles.jl (Section 2), present results on benchmark problems, two biological applications, and a comparison with NSGA-II (Section 3), discuss the findings in the context of the broader multiobjective optimization landscape (Section 4), and conclude with a summary and outlook (Section 5).

2 Methods

2.1 Pareto dominance and ranking

Consider an archive of n solutions evaluated on m objectives, where lower values are preferred. Solution $\mathbf{x}_j$ *strictly dominates* solution $\mathbf{x}_i$, written $\mathbf{x}_j \prec \mathbf{x}_i$, if and only if $f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i)$ for all $k \in \{1, \ldots, m\}$ and there exists at least one objective k for which $f_k(\mathbf{x}_j) < f_k(\mathbf{x}_i)$ (Equation (1)). The *Pareto rank* of solution $\mathbf{x}_i$ is the number of archive members that strictly dominate it (Equation (2)), so that solutions with rank zero lie on the Pareto front and identical solutions — which do not dominate each other under strict dominance — share the same rank. It is worth noting that the original POETs implementation [Song et al., 2010, Bassen et al., 2017] used *weak* dominance, checking only that $f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i)$ for all k without requiring strict inequality in at least one objective; under weak dominance, identical solutions count as dominating each other, which

inflates their ranks and causes excessive pruning of good solutions from the archive. The current implementation enforces strict Pareto dominance, and the pairwise test iterates over objectives with an early-exit condition that terminates as soon as any objective of j is found to be worse than i, avoiding unnecessary comparisons (Algorithm 1).

$$f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i) \quad \forall\, k \in \{1, \ldots, m\} \qquad \text{and} \qquad \exists\, k : f_k(\mathbf{x}_j) < f_k(\mathbf{x}_i). \tag{1}$$

$$R(\mathbf{x}_i) = \big|\{\, j \neq i \mid \mathbf{x}_j \prec \mathbf{x}_i \,\}\big|. \tag{2}$$

2.2 Incremental ranking

A central computational concern is the cost of ranking. When a single candidate solution $\mathbf{x}_{\text{new}}$ is appended to an archive of $n-1$ existing solutions with known ranks $\mathbf{R}$, the naïve approach recomputes all $n(n-1)$ pairwise dominance checks at $O(n^2 m)$ cost. Because only one solution changed, however, we can instead perform a single $O(nm)$ pass over the existing archive: for each existing solution $\mathbf{x}_i$, we test dominance in both directions, incrementing R_i if the new solution dominates $\mathbf{x}_i$ and accumulating toward the new solution's rank if $\mathbf{x}_i$ dominates it. This incremental procedure (Algorithm 2) reduces the per-iteration cost from $O(n^2 m)$ to $O(nm)$, with a full $O(n'^2 m)$ re-rank performed only after archive pruning when n' is small.

2.3 Pareto simulated annealing

The complete Pareto simulated annealing procedure combines the dominance check and incremental ranking into a single outer-inner loop structure (Algorithm 3). The algorithm is seeded with an initial parameter vector $\mathbf{x}_0$, an initial temperature $T = 1$, and an archive containing the single evaluated solution $(\mathbf{x}_0, \mathbf{f}(\mathbf{x}_0))$. At each temperature level, N_{iter} candidate solutions are generated by perturbing the current best parameter vector through a user-supplied neighbor function, and the rank array is updated incrementally via RankInsert at $O(nm)$ cost per candidate. The acceptance probability, computed from the rank array and the current temperature, determines whether the candidate is retained; upon acceptance, solutions whose rank meets or exceeds the cutoff R_{cutoff} are pruned and a full re-rank is performed on the smaller remaining set, with a hard size cap $n_{\max}$ enforced if the archive still exceeds the limit. Upon rejection, the candidate is immediately removed and the rank array is restored from a saved snapshot, a strategy we refer to as *pop-on-reject*; in the original implementation, rejected candidates remained in the archive, causing unbounded growth of n between accepted moves and progressively more expensive ranking operations, and the pop-on-reject strategy eliminates this problem entirely.

2.4 Parallel execution

Two opportunities for parallelism arise from the structure of the algorithm. First, the outer loop of both the full ranking and the incremental RankInsert procedure iterates independently over archive solutions, so setting `parallel_evaluation=true` dispatches to threaded variants that distribute these independent iterations across available threads using Julia's `Threads.@threads` construct, with an atomic accumulator used for R_{new} in the incremental case since multiple threads contribute to it. Second, and more significantly, because simulated annealing is inherently sequential within a single chain, the most effective parallelization strategy is to run C independent chains from different starting points and merge the resulting archives (Algorithm 4); each chain receives its own deterministic random number generator seeded from a base seed and the chain index, ensuring reproducibility, and after all chains complete, the archives are concatenated and a final full ranking is performed on the merged set.

2.5 Software design

ParetoEnsembles.jl is a pure Julia package with no dependencies beyond the standard library module `Random`, and its public API consists of five functions (Table 1). The primary entry point, `estimate_ensemble`, runs a single Pareto SA chain and returns the retained objective values, parameter vectors, and Pareto ranks; its parallel counterpart, `estimate_ensemble_parallel`, runs C independent chains concurrently, merges their archives, and re-ranks the combined set. A standalone `rank_function` computes Pareto ranks for an arbitrary $m \times n$ objective matrix and can be used independently of the optimization loop, while `hypervolume` computes the hypervolume indicator for a 2D front using a sweep-line algorithm in $O(n \log n)$ time and `pareto_front` extracts the rank-zero subset from the archive for downstream analysis.

The algorithm is entirely defined by four user-supplied callback functions (Table 2): an objective function that maps a parameter vector $\mathbf{x} \in \mathbb{R}^d$ to an $m \times 1$ array of objective values, a neighbor function that perturbs a given parameter vector to generate a candidate, an acceptance probability function that returns a scalar probability from the rank array and temperature, and a cooling function that maps the current temperature to a reduced value. Beyond these four callbacks, the behavior of `estimate_ensemble` is controlled by keyword arguments including the pruning threshold `rank_cutoff` (default 5), the number of candidates per temperature level `maximum_number_of_iterations` (default 20), the stopping temperature `temperature_min` (default 10^{-4}), and the hard archive size cap `maximum_archive_size` (default 1000). When `trace=true`, the algorithm records convergence diagnostics — temperature, archive size, and hypervolume — at each cooling step, returning a fourth element in the output tuple that allows users to monitor convergence and assess whether the annealing schedule was sufficient. Representative Julia code demonstrating both the single-chain and multi-chain interfaces is provided in the Supplementary Material (Listings 1 and 2).

3 Results

3.1 Standard benchmarks

We first demonstrate ParetoEnsembles.jl on two standard multiobjective benchmarks: the constrained Binh–Korn problem [Binh and Korn, 1997] (Equation (3)) and the unconstrained Fonseca–Fleming problem [Fonseca and Fleming, 1995] with $d = 3$ decision variables (Equation (4)). Constraints in the Binh–Korn problem are enforced via quadratic penalty terms added to the objective values, and bound constraints are handled by clamping in the neighbor function. We ran both benchmarks using ten parallel chains with $N_{\text{iter}} = 60$, $R_{\text{cutoff}} = 12$, and $\alpha = 0.95$, and the resulting Pareto fronts in objective space recover the characteristic trade-off curves for both problems, with a visible cloud of near-optimal solutions surrounding the non-dominated front (Figure 1a,b). The computed front for the Fonseca–Fleming case lies directly on the theoretical curve, confirming that the algorithm converges to the correct solution set, and in parameter space the Binh–Korn Pareto-optimal decision vectors trace a curved manifold from the origin toward $(5, 3)$ while the Fonseca–Fleming solutions cluster along the diagonal $x_1 \approx x_2$ as expected from the symmetric structure of the objectives (Figure 1c,d). Comparing a single chain against ten parallel chains on the Binh–Korn problem illustrates the benefit of multi-chain execution (Figure 2): the single chain recovers the trade-off curve but, because every retained solution is non-dominated within that chain's archive, the rank array contains only rank-zero entries, whereas the merged multi-chain run

produces a denser front with broader coverage of the feasible region.

$$\begin{aligned} \min_{\mathbf{x}} \quad & f_1 = 4x_1^2 + 4x_2^2, \qquad f_2 = (x_1-5)^2 + (x_2-5)^2 \\ \text{s.t.} \quad & (x_1-5)^2 + x_2^2 \le 25, \qquad (x_1-8)^2 + (x_2-3)^2 \ge 7.7 \\ & 0 \le x_1 \le 5, \quad 0 \le x_2 \le 3. \end{aligned} \tag{3}$$

$$f_1(\mathbf{x}) = 1 - \exp\left(-\sum_{i=1}^{d}\left(x_i - \tfrac{1}{\sqrt{d}}\right)^2\right), \qquad f_2(\mathbf{x}) = 1 - \exp\left(-\sum_{i=1}^{d}\left(x_i + \tfrac{1}{\sqrt{d}}\right)^2\right). \tag{4}$$

3.2 Cell-free gene expression

As a first biological application, we considered the estimation of kinetic parameters for a cell-free gene expression circuit describing the production of deGFP protein driven by the endogenous sigma factor σ_{70} via the P70 promoter, following the effective biophysical framework of Adhikari et al. [2020]. The system is governed by two ordinary differential equations for mRNA (m) and protein (p), where $\dot{m} = \alpha\, u(\sigma_{70}) - \delta_m\, m$ and $\dot{p} = \kappa\, m\, w(t) - \delta_p\, p$, with $u(\sigma_{70}) = \sigma_{70}^n/(K^n + \sigma_{70}^n)$ representing a Hill-type promoter activity function and $w(t) = \exp(-\ln 2 \cdot t/\tau_{1/2})$ capturing the experimentally observed decay of translational capacity over the course of the cell-free reaction. Five parameters were treated as unknown — the maximum transcription rate α, the effective translation rate constant κ, the mRNA degradation rate δ_m, the promoter dissociation constant K, and the translation capacity half-life $\tau_{1/2}$ — while the remaining parameters ($\sigma_{70} = 35$ nM, $n = 1.5$, $\delta_p = 0.005$ h^{-1}) were fixed at values taken from the literature.

We fitted the model to experimental data from Adhikari et al. [2020], consisting of mRNA and deGFP protein concentrations measured at five time points (0, 2, 4, 8, and 16 h) in triplicate, with reported means and standard deviations. Two objectives were defined as standard-deviation-weighted sums of squared errors: $\varepsilon_{\text{mRNA}} = \sum_i [(m_i^{\text{sim}} - m_i^{\text{obs}})/\sigma_i^m]^2$ and $\varepsilon_{\text{protein}} = \sum_i [(p_i^{\text{sim}} - p_i^{\text{obs}})/\sigma_i^p]^2$, and we ran ten parallel chains with $R_{\text{cutoff}} = 8$ and $N_{\text{iter}} = 50$, starting from random initial parameter vectors drawn uniformly within the bounds. The resulting ensemble captured the dynamics of both mRNA and protein expression (Figure 3a,b): the 95% confidence interval of the ensemble simulations bracketed the experimental data at all time points, and the ensemble mean tracked the observed trajectories. The Pareto front revealed a clear trade-off between fitting the mRNA and protein time courses (Figure 3c), with parameter sets that minimized $\varepsilon_{\text{mRNA}}$ tending to have slightly larger $\varepsilon_{\text{protein}}$ and vice versa — a trade-off that arises naturally because the mRNA degradation rate and translation capacity half-life have opposing effects on the two species, so that the ensemble provides a family of plausible parameter sets rather than a single point estimate, enabling downstream uncertainty quantification of model predictions.

3.3 Blood coagulation cascade

To demonstrate the package on a larger-scale system, we considered the Hockin–Mann model of tissue-factor-initiated blood coagulation [Hockin et al., 2002], a mechanistic ODE model describing 34 chemical species interacting through 27 reactions governed by 42 rate constants that captures the initiation, amplification, and inhibition phases of thrombin generation. All rate constants are available in the literature and were used as ground truth, and we selected ten key catalytic and inhibition rate constants for estimation spanning the extrinsic factor Xase (k_{cat}), intrinsic factor Xase (k_{cat}), prothrombinase (k_{cat}), and antithrombin III inhibition pathways. Because these rate constants span six orders of magnitude (from $\sim$1 to $\sim 10^7$), we worked in log-space with bounds of ± 1.5 orders of magnitude around the true values, and generated noisy "experimental" data by

simulating the model with the true rate constants at two tissue factor (TF) concentrations (5 pM and 25 pM) and adding 10% multiplicative Gaussian noise to the total thrombin trajectory. Three objectives were defined: the normalized sum-of-squared error of the thrombin trajectory at 5 pM TF, the normalized SSE at 25 pM TF, and a regularization term measuring the mean squared log-distance of estimated rate constants from their literature values.

We ran eight parallel chains with $R_{\text{cutoff}} = 10$, $N_{\text{iter}} = 40$, and $\alpha = 0.92$, and the resulting ensemble of 140 parameter sets (rank ≤ 1) produced thrombin trajectories that closely matched the noisy data at both TF concentrations (Figure 4a,b). Parameter recovery was strong: median estimates for 8 of 10 rate constants fell within 10% of the true values, with the prothrombinase k_{cat} and antithrombin inhibition rates recovered to within 1–5% (Figure 4c), while the hardest parameter to recover was the Factor VIIIa activation rate ($k_{\text{IIa}\to\text{VIIIa}}$, 45% error), reflecting its weaker influence on the thrombin trajectory relative to the dominant prothrombinase pathway. The three-dimensional Pareto front (Figure 4d) revealed the expected trade-off between fitting the two TF conditions and staying close to the literature values, demonstrating that ParetoEnsembles.jl scales effectively to systems with tens of species, ten or more estimated parameters, and three competing objectives.

3.4 Comparison with NSGA-II

To position ParetoEnsembles.jl relative to population-based evolutionary methods, we compared it against NSGA-II [Deb et al., 2002] as implemented in Metaheuristics.jl [Mejía-de Dios et al., 2022] on both benchmark problems, matching the function evaluation budget at approximately 110,000 evaluations per solver (ParetoEnsembles: 10 chains, $N_{\text{iter}} = 60$, $\alpha = 0.95$; NSGA-II: population 200, 550 generations). Five independent replicates were run for each solver, and the median hypervolume indicator (HV) and inverted generational distance (IGD) are reported in Table 4, with IGD computed against the analytical Pareto front for the Fonseca–Fleming problem and against a densely sampled reference front from a high-budget run for Binh–Korn. ParetoEnsembles.jl achieved comparable or slightly higher hypervolume than NSGA-II on both problems and substantially lower IGD — 5× lower on Binh–Korn and 7× lower on Fonseca–Fleming — indicating a closer approximation to the true Pareto front (Figure 5), while also producing much denser fronts with 2,000–6,000 non-dominated solutions compared with 200 for NSGA-II, reflecting its design as an ensemble generator that characterizes the full cloud of near-optimal solutions rather than merely approximating the front boundary. NSGA-II was 50–150× faster in wall time, consistent with its population-parallel evaluation strategy, but this speed advantage comes at the cost of a sparser representation of the trade-off surface that may be insufficient for downstream uncertainty analysis.

3.5 Hyperparameter sensitivity

We assessed the sensitivity of ParetoEnsembles.jl to three key hyperparameters — rank cutoff R_{cutoff}, cooling rate α, and iterations per temperature N_{iter} — by sweeping each while holding the others at default values ($R_{\text{cutoff}} = 8$, $\alpha = 0.90$, $N_{\text{iter}} = 40$) on the Binh–Korn benchmark with five chains and five replicates per configuration (Figure 6). The hypervolume indicator was robust across all tested ranges: rank cutoff values from 2 to 12 produced nearly identical results, with a modest decline at $R_{\text{cutoff}} = 20$ where the archive retains many low-quality solutions; faster cooling ($\alpha = 0.80$) performed comparably to slower cooling ($\alpha = 0.95$), with the latter showing slightly more variability; and even 10 iterations per temperature level produced competitive fronts, though 20 or more reduced inter-replicate variance. These results suggest that the default hyper-

parameters provide a reasonable starting point for most problems, and that users need not invest significant effort in tuning — a practical advantage for the systems biology applications where ParetoEnsembles.jl is primarily intended to be used.

3.6 Parallel scaling

To quantify the benefit of multi-chain parallelism, we measured wall clock time for the cell-free ensemble estimation (ten chains, $N_{\text{iter}} = 50$) as a function of the number of Julia threads (Table 3). Each configuration was run five times and the median time is reported; the speedup is approximately linear up to four threads (3.1×) and reaches 4.1× at eight threads, with diminishing returns due to load imbalance when ten chains are distributed across eight threads and the serial cost of the final archive merge and re-ranking step.

4 Discussion

ParetoEnsembles.jl provides a lightweight Julia implementation of Pareto Optimal Ensemble Techniques for multiobjective optimization that addresses several limitations of the original POETs algorithm [Song et al., 2010, Bassen et al., 2017]. The current version corrects the dominance relation from weak to strict, reduces the per-iteration ranking cost from $O(n^2m)$ to $O(nm)$ through incremental updating, bounds archive growth through the pop-on-reject strategy and a hard size cap, adds both threaded ranking and multi-chain parallel execution, and provides built-in hypervolume computation and convergence diagnostics. The callback-based design decouples the optimization algorithm from the problem definition, allowing users to apply ParetoEnsembles.jl to any domain where objective functions can be evaluated pointwise, and the head-to-head comparison with NSGA-II demonstrates that this SA-based approach achieves comparable hypervolume and substantially better IGD while producing fronts that are 10–30× denser than those returned by a leading evolutionary method at matched evaluation budgets. While NSGA-II is considerably faster in wall time — reflecting the inherently sequential nature of SA chains versus the population-parallel evaluation of evolutionary methods — ParetoEnsembles.jl targets problems where the primary goal is to characterize the *cloud* of near-optimal parameter sets, not just the front itself, and where the simplicity of the interface lowers the barrier to generating parameter ensembles as part of a modeling workflow.

The biological applications presented here illustrate both the strengths and the current scope of the package. The cell-free gene expression example demonstrates that even a small model with five parameters and two objectives can reveal meaningful trade-offs when fitted to real experimental data, and the coagulation cascade example shows that the approach scales to systems with 34 species, ten estimated parameters spanning six orders of magnitude, and three competing objectives, recovering most rate constants to within 10% of their true values. The hyperparameter sensitivity analysis further shows that the algorithm is robust to the choice of rank cutoff, cooling rate, and iterations per temperature, so that users can adopt the default settings without extensive tuning. The convergence trace feature, activated via the `trace` keyword argument, provides a built-in diagnostic for monitoring archive growth and hypervolume evolution over the annealing schedule, helping users assess whether the algorithm has run long enough for their particular problem.

Several features are deliberately omitted from the current scope. The package does not implement gradient-based refinement, surrogate modeling, or adaptive cooling schedules, nor does it provide built-in visualization or benchmark problem libraries. Integration with Julia's automatic differentiation ecosystem, for example through ForwardDiff.jl or Enzyme.jl, could enable hybrid gradient-free/gradient-based strategies for problems where gradients are available for some but not

all objectives, and adaptive archive management using hypervolume indicators [Zitzler et al., 2003] in place of simple rank-based pruning would provide more principled control over the distribution of retained solutions along the Pareto front.

5 Conclusions

We have presented ParetoEnsembles.jl, a Julia package for generating Pareto-optimal parameter ensembles using simulated annealing with incremental dominance ranking. The package corrects a longstanding weak-dominance bug in the original POETs implementation, reduces the per-iteration ranking cost from quadratic to linear in the archive size, and adds multi-chain parallel execution that provides improved front coverage from diverse starting points. On standard benchmarks, ParetoEnsembles.jl produces fronts with hypervolume and inverted generational distance that are competitive with or superior to NSGA-II at matched evaluation budgets, while retaining ensembles that are 10–30× denser — a feature that is particularly valuable for downstream uncertainty quantification in parameter estimation problems. The package was applied to two biological systems of increasing complexity: a cell-free gene expression circuit fitted to experimental mRNA and protein time-course data, and a blood coagulation cascade model with 34 species and ten estimated rate constants spanning six orders of magnitude, where the ensemble recovered most parameters to within 10% of their true values. ParetoEnsembles.jl is open source, registered in the Julia General registry, and can be installed with a single command; we anticipate that its minimal interface and dependency-free design will make it a useful tool for researchers who need to generate parameter ensembles for mechanistic models in systems biology and beyond.

Acknowledgments

J.D.V. acknowledges support from the National Heart, Lung, and Blood Institute (NHLBI) of the National Institutes of Health (NIH) under awards R33 HL141787 and R01 HL71944.

Author Contributions

J.D.V. conceived the study, developed the software, performed the analyses, and wrote the manuscript.

Competing Interests

The author declares no competing interests.

Data and Code Availability

ParetoEnsembles.jl is open source under the MIT license. The source code is available at `https://github.com/varnerlab/ParetoEnsembles.jl`, and documentation is hosted at `https://varnerlab.github.io/ParetoEnsembles.jl/dev/`. The package is registered in the Julia General registry and can be installed via `Pkg.add("ParetoEnsembles")`. Example code reproducing all results in this paper is included in the `paper/code` directory of the repository.

References

Abhinav Adhikari, Michael Vilkhovoy, Sandra Vadhin, Ha Eun Lim, and Jeffrey D. Varner. Effective biophysical modeling of cell free transcription and translation processes. *Frontiers in Bioengineering and Biotechnology*, 8:539081, 2020. doi: 10.3389/fbioe.2020.539081.

Sanghamitra Bandyopadhyay, Sriparna Saha, Ujjwal Maulik, and Kalyanmoy Deb. A simulated annealing-based multiobjective optimization algorithm: AMOSA. *IEEE Transactions on Evolutionary Computation*, 12(3):269–283, 2008. doi: 10.1109/TEVC.2007.900837.

David Bassen, Michael Vilkhovoy, Mason Minot, Jonathan Butcher, and Jeffrey D. Varner. JuPOETs: A constrained multiobjective optimization approach to estimate biochemical model ensembles in the Julia programming language. *BMC Systems Biology*, 11(1):10, 2017. doi: 10.1186/s12918-016-0380-2.

Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi: 10.1137/141000671.

To Thanh Binh and Ulrich Korn. MOBES: A multiobjective evolution strategy for constrained optimization problems. In *Proceedings of the Third International Conference on Genetic Algorithms (Mendel 97)*, pages 176–182, 1997.

Julian Blank and Kalyanmoy Deb. pymoo: Multi-objective optimization in Python. *IEEE Access*, 8:89497–89509, 2020. doi: 10.1109/ACCESS.2020.2990567.

Carlos A. Coello Coello, Gary B. Lamont, and David A. Van Veldhuizen. *Evolutionary Algorithms for Solving Multi-Objective Problems*. Springer, second edition, 2007. doi: 10.1007/978-0-387-36797-2.

Piotr Czyżak and Andrzej Jaszkiewicz. Pareto simulated annealing—a metaheuristic technique for multiple-objective combinatorial optimization. *Journal of Multi-Criteria Decision Analysis*, 7 (1):34–47, 1998. doi: 10.1002/(SICI)1099-1360(199801)7:1⟨34::AID-MCDA161⟩3.0.CO;2-6.

Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons, 2001.

Kalyanmoy Deb and Himanshu Jain. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18(4):577–601, 2014. doi: 10.1109/TEVC.2013.2281535.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2): 182–197, 2002. doi: 10.1109/4235.996017.

Carlos M. Fonseca and Peter J. Fleming. An overview of evolutionary algorithms in multiobjective optimization. *Evolutionary Computation*, 3(1):1–16, 1995. doi: 10.1162/evco.1995.3.1.1.

Ryan N. Gutenkunst, Joshua J. Waterfall, Fergal P. Casey, Kevin S. Brown, Christopher R. Myers, and James P. Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS Computational Biology*, 3(10):e189, 2007. doi: 10.1371/journal.pcbi.0030189.

Matthew F. Hockin, Kenneth C. Jones, Scott J. Everse, and Kenneth G. Mann. A model for the stoichiometric regulation of blood coagulation. *Journal of Biological Chemistry*, 277(21): 18322–18333, 2002. doi: 10.1074/jbc.M201173200.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

Jesús-Adolfo Mejía-de Dios, Efrén Mezura-Montes, and Carlos A. Coello Coello. Metaheuristics.jl: Towards reproducible optimization research. *Journal of Open Source Software*, 7(78):4723, 2022. doi: 10.21105/joss.04723.

Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Springer, 1999. doi: 10.1007/978-1-4615-5563-6.

Satyaprakash Song, Anirikh Chakrabarti, Jay Bhatt, Devendranath Bhatt, and Jeffrey D. Varner. Identifying ensembles of signal transduction models using Pareto optimal ensemble techniques (POETs). *Biotechnology Journal*, 5(7):768–780, 2010. doi: 10.1002/biot.201000059.

Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007. doi: 10.1109/TEVC.2007.892759.

Eckart Zitzler, Lothar Thiele, Marco Laumanns, Carlos M. Fonseca, and Viviane Grunert da Fonseca. Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2):117–132, 2003. doi: 10.1109/TEVC.2003.810758.

Algorithm 1 StrictlyDominates($\mathbf{f}_j, \mathbf{f}_i, m$) — Does solution j strictly dominate solution i?

Require: Objective vectors $\mathbf{f}_j, \mathbf{f}_i \in \mathbb{R}^m$ (lower is better)
Ensure: TRUE if $\mathbf{f}_j \prec \mathbf{f}_i$, FALSE otherwise

```
strictly_better ← FALSE
for k = 1, ..., m do
    if f_{j,k} > f_{i,k} then
        return FALSE                          ▷ j is worse in objective k; cannot dominate
    else if f_{j,k} < f_{i,k} then
        strictly_better ← TRUE                ▷ j is better in at least one objective
    end if
end for
return strictly_better
```

Algorithm 2 RankInsert($\mathcal{A}, \mathbf{R}$) — Incremental rank update after appending one solution

Require: Archive $\mathcal{A} = \{(\mathbf{x}_1, \mathbf{f}_1), \ldots, (\mathbf{x}_n, \mathbf{f}_n)\}$ where $(\mathbf{x}_n, \mathbf{f}_n)$ is newly appended
Require: Rank array $\mathbf{R} = [R_1, \ldots, R_{n-1}]$ valid for solutions $1, \ldots, n-1$
Ensure: Updated $\mathbf{R} = [R_1, \ldots, R_n]$ valid for the full archive

```
R_new ← 0
for i = 1, ..., n − 1 do
    if StrictlyDominates(f_n, f_i, m) then
        R_i ← R_i + 1                         ▷ new solution dominates i
    end if
    if StrictlyDominates(f_i, f_n, m) then
        R_new ← R_new + 1                     ▷ i dominates new solution
    end if
end for
Append R_new to R
```

Table 1: Public API of ParetoEnsembles.jl.

Function	Description
`estimate_ensemble`	Run a single Pareto SA chain. Returns objective values, parameter vectors, and Pareto ranks for the retained archive.
`estimate_ensemble_parallel`	Run C independent chains in parallel (one per starting point), merge archives, and re-rank.
`rank_function`	Compute Pareto ranks for an $m \times n$ objective matrix.
`hypervolume`	Compute the hypervolume indicator for a 2D Pareto front given a reference point.
`pareto_front`	Extract Pareto-optimal (rank = 0) solutions from the archive.

Algorithm 3 ParetoEnsembles — Pareto simulated annealing with incremental ranking

Require: User callbacks: Objective, Neighbor, Accept, Cool
Require: Initial parameter vector $\mathbf{x}_0$; hyperparameters N_{iter}, R_{cutoff}, $T_{\min}$, $n_{\max}$
Ensure: Archive of solutions with objective values, parameters, and Pareto ranks

$\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}_0$; $\mathcal{A} \leftarrow \{(\mathbf{x}_0,\ \textsc{Objective}(\mathbf{x}_0))\}$; $\mathbf{R} \leftarrow [0]$; $T \leftarrow 1$

while $T > T_{\min}$ **do**
  for $t = 1, \ldots, N_{\text{iter}}$ **do**
    $\mathbf{x}' \leftarrow \textsc{Neighbor}(\mathbf{x}_{\text{best}})$; $\mathbf{f}' \leftarrow \textsc{Objective}(\mathbf{x}')$
    $\mathbf{R}_{\text{saved}} \leftarrow \text{copy}(\mathbf{R})$; Append $(\mathbf{x}', \mathbf{f}')$ to $\mathcal{A}$; $\textsc{RankInsert}(\mathcal{A}, \mathbf{R})$
    if $\textsc{Accept}(\mathbf{R}, T) > U(0,1)$ **then**
      $\mathcal{A} \leftarrow \{(\mathbf{x}_i, \mathbf{f}_i) \in \mathcal{A} \mid R_i < R_{\text{cutoff}}\}$; $\mathbf{R} \leftarrow \textsc{FullRank}(\mathcal{A})$
      if $|\mathcal{A}| > n_{\max}$ **then** Keep only the $n_{\max}$ solutions with lowest R_i; $\mathbf{R} \leftarrow \textsc{FullRank}(\mathcal{A})$
      end if
      $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}'$
    else
      Remove $(\mathbf{x}', \mathbf{f}')$ from $\mathcal{A}$; $\mathbf{R} \leftarrow \mathbf{R}_{\text{saved}}$
    end if
  end for
  $T \leftarrow \textsc{Cool}(T)$
end while
$\mathbf{R} \leftarrow \textsc{FullRank}(\mathcal{A})$; **return** $\mathcal{A}$, $\mathbf{R}$

Algorithm 4 ParetoEnsemblesParallel — Multi-chain parallel execution

Require: Initial states $\{\mathbf{x}_0^{(1)}, \ldots, \mathbf{x}_0^{(C)}\}$; base seed s
Ensure: Merged archive with Pareto ranks

for $c = 1, \ldots, C$ **in parallel do**
  $\text{rng}_c \leftarrow \textsc{Seed}(s, c)$; $(\mathcal{A}_c, \mathbf{R}_c) \leftarrow \textsc{ParetoEnsembles}(\mathbf{x}_0^{(c)}, \text{rng}_c)$
end for
$\mathcal{A} \leftarrow \mathcal{A}_1 \cup \cdots \cup \mathcal{A}_C$; $\mathbf{R} \leftarrow \textsc{FullRank}(\mathcal{A})$; **return** $\mathcal{A}$, $\mathbf{R}$

Table 2: User-supplied callback functions.

Callback	Signature	Purpose
`objective_function`	$(\mathbf{x}) \to \mathbb{R}^{m \times 1}$	Evaluate m objectives at parameter vector $\mathbf{x}$
`neighbor_function`	$(\mathbf{x}) \to \mathbb{R}^d$	Generate a candidate by perturbing $\mathbf{x}$
`acceptance_probability_function`	$(\mathbf{R}, T) \to [0, 1]$	Acceptance probability given ranks and temperature
`cooling_function`	$(T) \to T'$	Annealing schedule

Table 3: Multi-chain parallel scaling on the cell-free gene expression model (ten chains, $N_{\text{iter}} = 50$, median of five runs).

Threads	**Wall clock (s)**	**Speedup**
1	18.52	1.0×
2	9.38	2.0×
4	5.94	3.1×
8	4.51	4.1×

Table 4: Comparison of ParetoEnsembles.jl and NSGA-II on benchmark problems (median over 5 replicates). Higher HV and lower IGD are better. Both solvers used ∼110,000 function evaluations.

Problem	**Solver**	**HV**	**IGD**	**Front size**	**Time (s)**
Binh–Korn	ParetoEnsembles	**8035**	**0.014**	6172	77
	NSGA-II	8027	0.071	200	0.5
Fonseca–Fleming	ParetoEnsembles	**0.44**	**0.0004**	2051	3.1
	NSGA-II	0.44	0.003	200	0.5

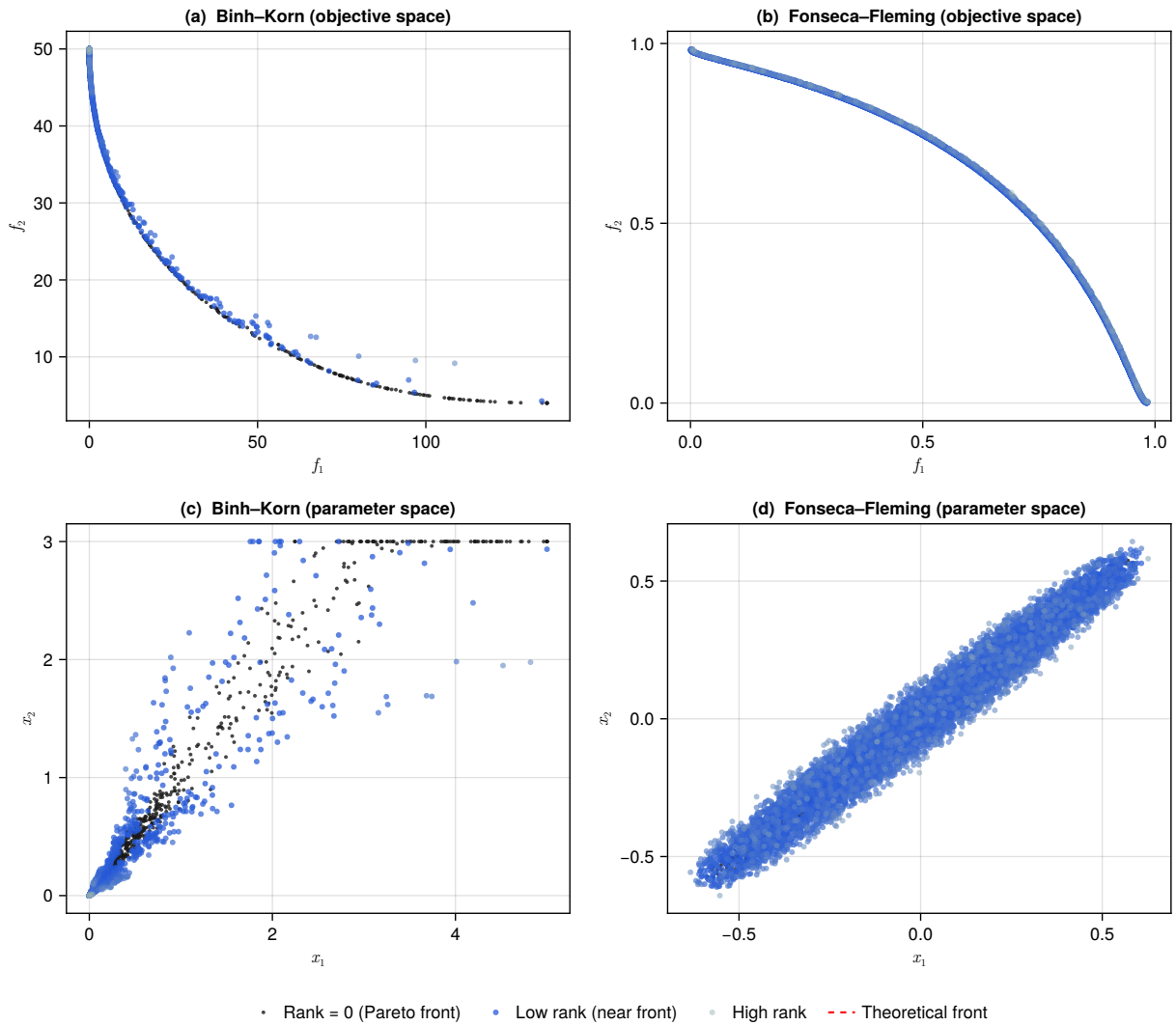

Figure 1: Benchmark results for (a,c) Binh–Korn and (b,d) Fonseca–Fleming ($d = 3$). Top row: objective space; bottom row: parameter-space projections (x_1 vs. x_2). Non-dominated solutions (rank $= 0$, dark) define the Pareto front, while near-optimal solutions (blue, colored by rank) form a cloud around it representing the retained ensemble. The dashed red curve in (b) is the theoretical Pareto front. Ten parallel chains were used with $R_{\text{cutoff}} = 12$, $N_{\text{iter}} = 60$, and $\alpha = 0.95$.

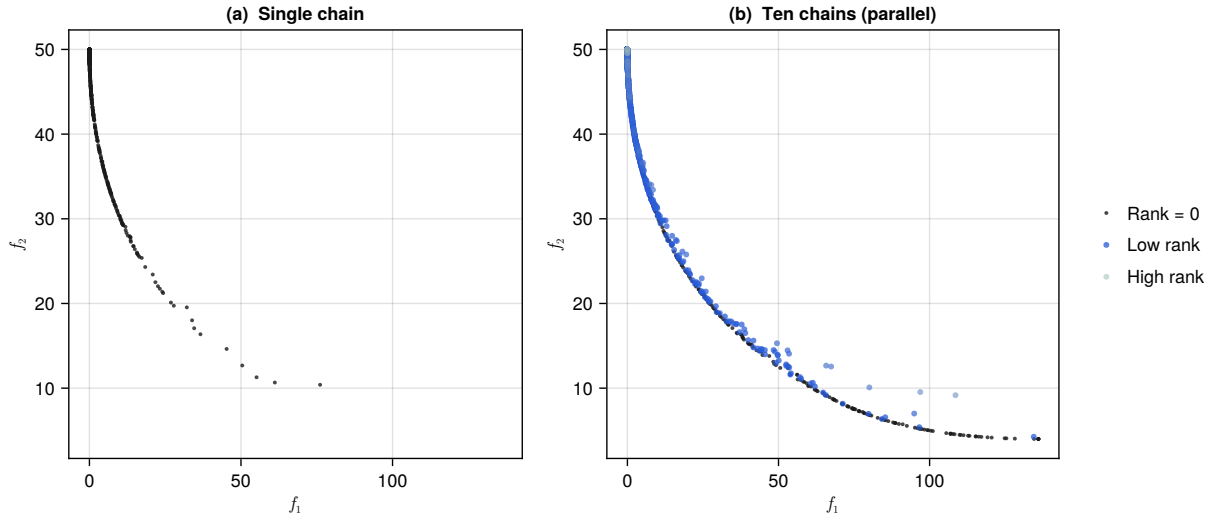

Figure 2: Comparison of (a) single-chain and (b) ten-chain parallel execution on the Binh–Korn problem. Both panels share the same axis limits. The single chain yields a sparse archive of non-dominated solutions, while the merged multi-chain archive provides denser front coverage and a visible cloud of near-optimal solutions (blue) surrounding the front.

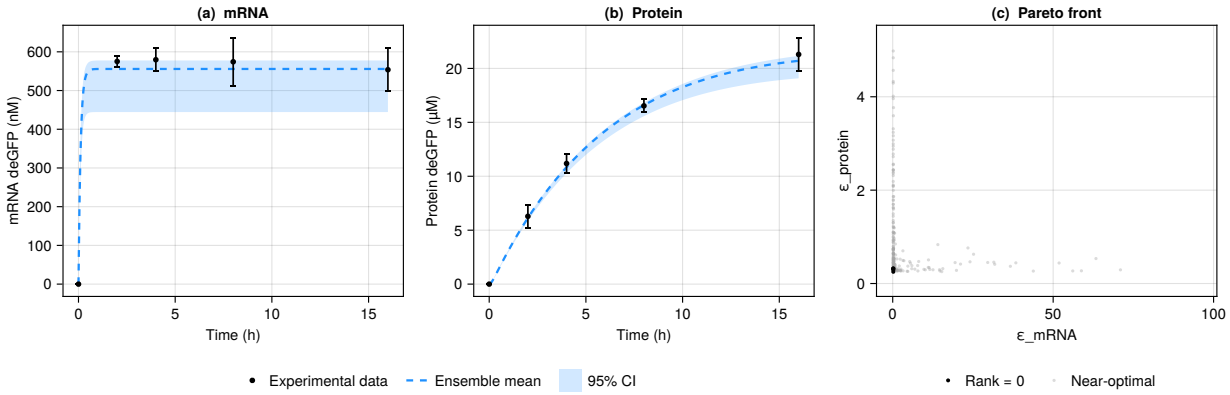

Figure 3: Ensemble estimation for a cell-free gene expression circuit ($\sigma_{70} \rightarrow$ P70 $\rightarrow$ deGFP) fitted to experimental data from Adhikari et al. [2020]. (a) mRNA and (b) protein concentration versus time; the blue dashed line is the ensemble mean, the shaded region is the 95% confidence interval, and black points are experimental data with error bars. (c) Pareto front showing the trade-off between mRNA and protein fitting error.

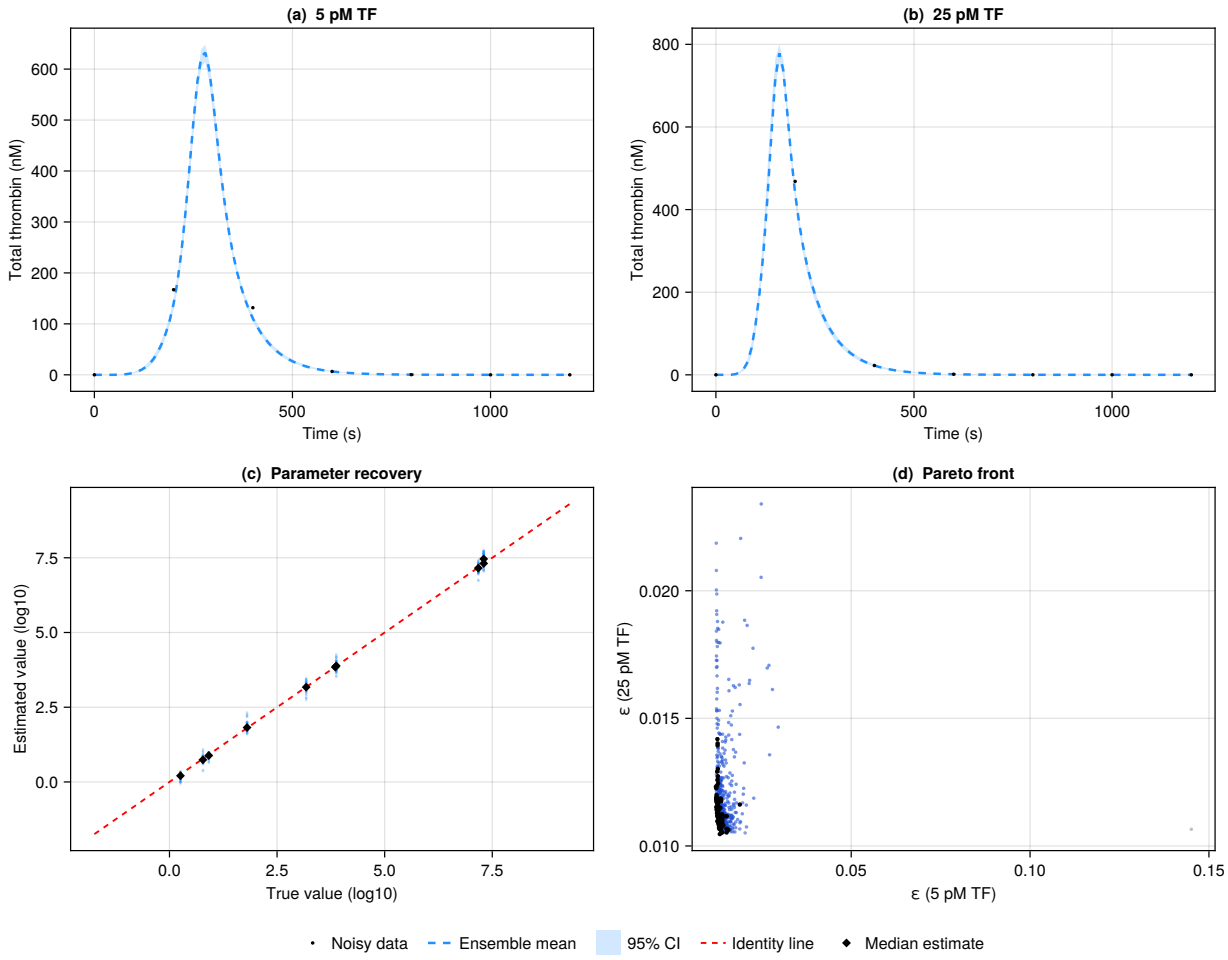

Figure 4: Ensemble estimation for the Hockin–Mann blood coagulation model (34 species, 10 estimated rate constants, 3 objectives). (a,b) Total thrombin concentration versus time at 5 pM and 25 pM tissue factor; the blue dashed line is the ensemble mean, the shaded region is the 95% CI, and black points are noisy data. (c) Parameter recovery: estimated versus true rate constants in log-space; the dashed red line is the identity and diamonds show the median ensemble estimate. (d) Pareto front projection (ε_1 vs. ε_2), with rank-zero solutions in black.

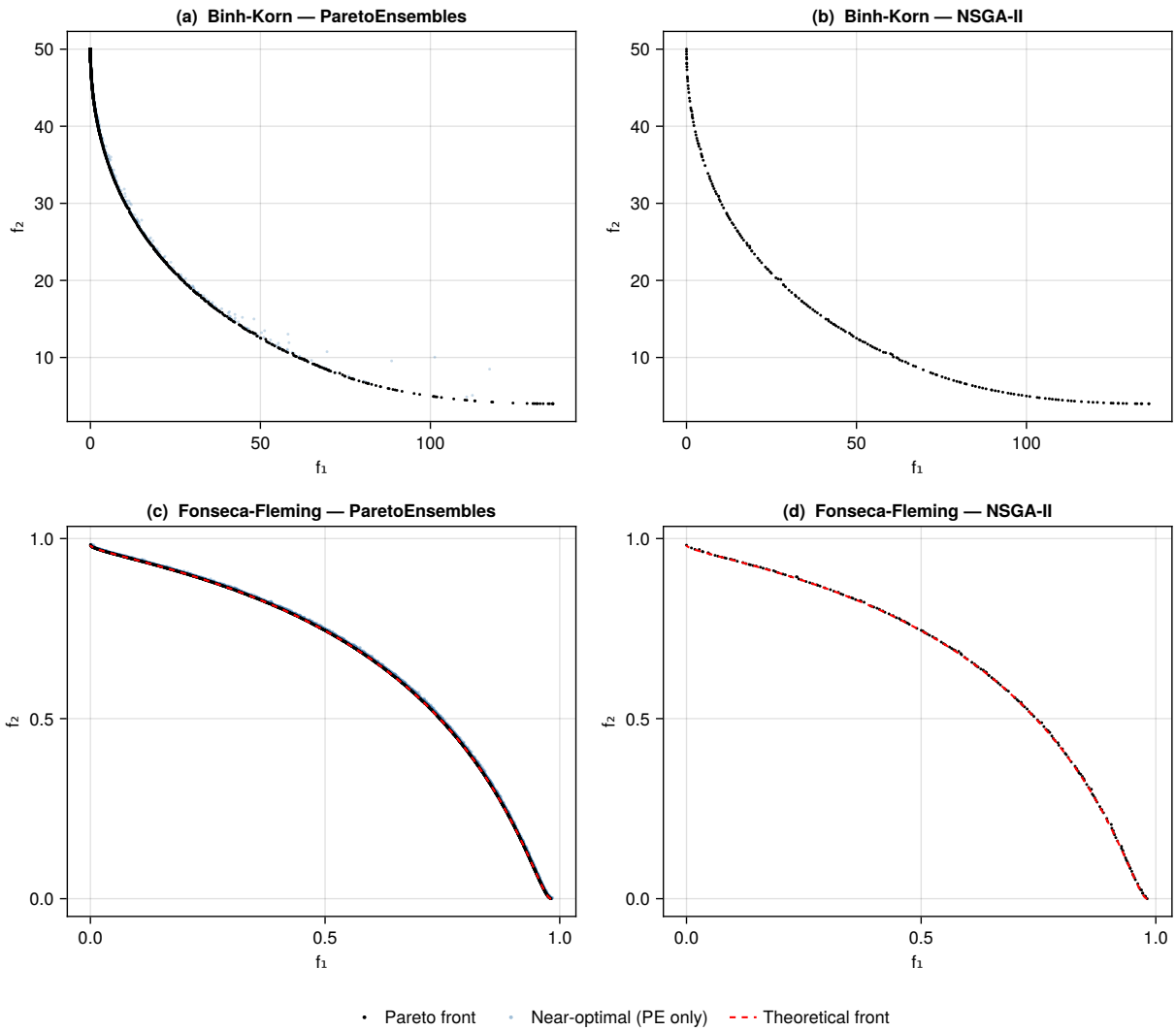

Figure 5: Comparison of ParetoEnsembles.jl (left) and NSGA-II (right) on (a,b) Binh–Korn and (c,d) Fonseca–Fleming at matched evaluation budgets (∼110,000). ParetoEnsembles.jl retains a cloud of near-optimal solutions (blue) in addition to the Pareto front (black), while NSGA-II returns only the non-dominated front. The dashed red curve in (c,d) is the theoretical Fonseca–Fleming front.

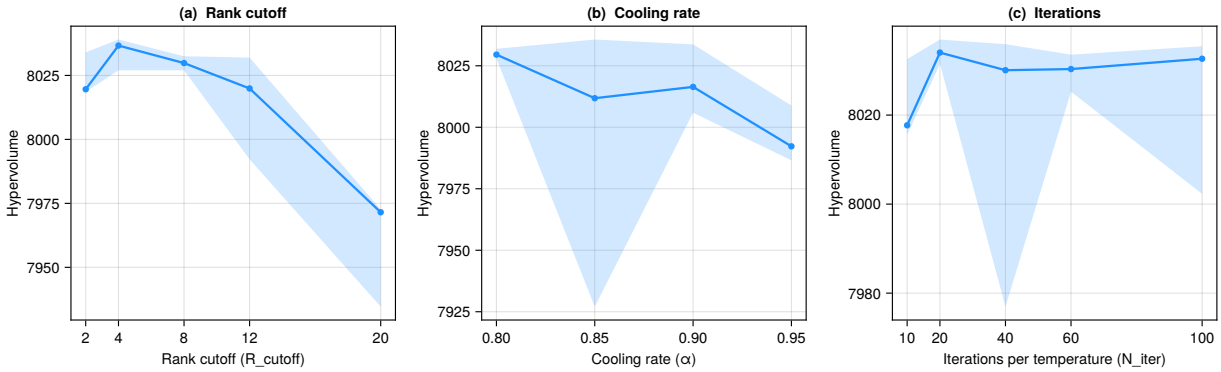

Figure 6: Hyperparameter sensitivity on the Binh–Korn benchmark. Median hypervolume (line) with interquartile range (shaded band) over 5 replicates. (a) Rank cutoff R_{cutoff}, (b) cooling rate α, (c) iterations per temperature N_{iter}. Default values for the other two parameters are held constant in each sweep.

Supplementary Material

Listing 1: Binh–Korn benchmark with ParetoEnsembles.jl.

```julia
using ParetoEnsembles

function objective_function(x)
    f = zeros(2, 1)
    f[1] = 4.0 * x[1]^2 + 4.0 * x[2]^2
    f[2] = (x[1] - 5)^2 + (x[2] - 5)^2
    # penalty for constraint violations
    lambda = 100.0
    v1 = 25 - (x[1] - 5)^2 - x[2]^2
    v2 = (x[1] - 8)^2 + (x[2] - 3)^2 - 7.7
    f[1] += lambda * (min(0, v1))^2
    f[2] += lambda * (min(0, v2))^2
    return f
end

neighbor_function(x) = clamp.(
    x .* (1 .+ 0.05 * randn(length(x))),
    [0.0, 0.0], [5.0, 3.0])

acceptance_probability_function(R, T) =
    exp(-R[end] / T)

cooling_function(T) = 0.9 * T

(EC, PC, RA) = estimate_ensemble(
    objective_function, neighbor_function,
    acceptance_probability_function, cooling_function,
    [2.5, 1.5]; rank_cutoff=4.0,
    maximum_number_of_iterations=40, show_trace=false)

pareto_idx = findall(RA .== 0)
```

Listing 2: Multi-chain parallel execution on the Binh–Korn problem (requires `julia -t4`).

```julia
using ParetoEnsembles

# ... (same callbacks as Listing 1) ...

initial_states = [
    [2.5, 1.5], [0.5, 2.5],
    [4.0, 0.5], [1.0, 1.0]
]

(EC, PC, RA) = estimate_ensemble_parallel(
    objective_function, neighbor_function,
    acceptance_probability_function, cooling_function,
    initial_states;
    rank_cutoff=4.0, maximum_number_of_iterations=40,
    show_trace=false, rng_seed=42)

```

```
println("Solutions: ", size(EC, 2))
println("Pareto-optimal: ", count(RA .== 0))
```